

MELSEC 通信 A P I (MCP、SLMP)

ユーザーズ・マニュアル

承認	照査	作成
		宮本



株式会社 ニコス

図番 NCS211920-M010

改版履歴

Rev1. 0. 0	2021/08/26	宮本	初版作成
Rev1. 1. 0	2021/10/04	宮本	API 初期化パラメータ、ログ書込みコールバック関数変更
Rev1. 2. 0	2021/12/27	宮本	MELSEC_DevStrCs、MELSEC_DevCs の追加 MELSEC_DEV_INFO 構造体変更
Rev1. 3. 0	2022/01/20	宮本	SLMP ラベル名指定 API の追加
Rev1. 4. 0	2022/04/27	宮本	コールバック関数説明の変更・追記

□□□ 目 次 □□□

1. はじめに	4
2. API関数仕様	6
2-1. APIの初期化／解放	7
MELSEC_GetVer	8
MELSEC_Initialize	9
MELSEC_Release	17
MELSEC_ReleaseAll	18
2-2. 通信回線のオープン、接続、クローズ	19
MELSEC_Open	20
MELSEC_Connect	25
MELSEC_Close	29
MELSEC_IsOpen	30
2-3. MELSECとの通信	33
MELSEC_DevStr	34
MELSEC_DevStrCs	35
MELSEC_Dev	38
MELSEC_DevCs	38
MELSEC_GetDevInfo	40
MELSEC_GetComParm	42
MELSEC_ReadBits	47
MELSEC_ReadWords	49
MELSEC_WriteBits	50
MELSEC_WriteWords	51
MELSEC_RemoteCpu	52
MELSEC_GetCpuType	53
MELSEC_LoopbackTest	54
MELSEC_ReadArrayLabel	55
MELSEC_WriteArrayLabel	56

1. はじめに

本ユーザーズ・マニュアルは、MELSEC通信API（DLL）仕様について記述したものです。
 ニコス設計アプリケーション内の三菱電機社製PLC（MELSEC）との通信プログラムは、オーダー単位・設計者単位で異なっているために、本通信APIを使用して標準化を行う。

MELSEC通信APIは、Microsoft社製Visual StudioのC/C++、C#、VB言語で開発されたアプリケーションとリンクすることができる。

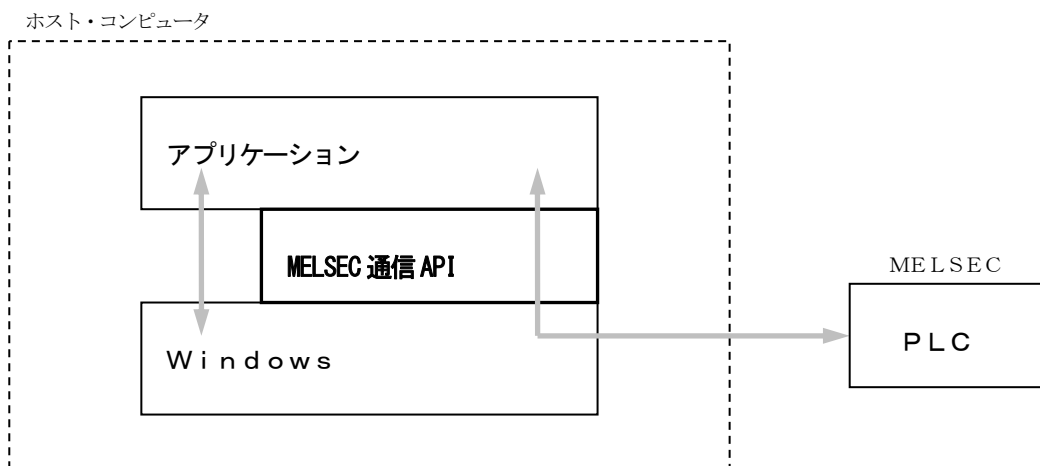
MELSEC通信APIは、ヘッダーファイル・ライブラリファイル・DLLファイルで提供され、アプリケーション・プロジェクトに静的にライブラリを登録して使用する。C#・VBでアプリケーションを開発する場合は、DLLファイルを動的にインポートして使用する。

MELSEC通信APIは、最大100個のインスタンスで管理することができる。

【システムログAPI配布ファイル】

ファイル種類	ターゲット	文字セット	ファイル名	開発環境			備考
				C++	C#	VB	
ヘッダーファイル	共通	共通	MELSEC_API.h	○			C++時のみ使用
ライブラリファイル	32bit	無し	MELSEC_API.lib	○			C++時のみ使用
	64bit						
DLLファイル	32bit	無し	MELSEC_API.dll	○	○	○	
	64bit						

ファイル名の**太字**ファイル名は、32ビットターゲットで文字セット無し時の使用ファイルを表す。



MELSECとの通信は、専用プロトコルMCP (Melsec Communication Protocol) およびSLMP (SeamLess Message Protocol) で行う。専用プロトコルは、下記の種類に分類される。

通信媒体	伝送フレーム	通信形式	備考
シリアル	A 互換 1C フレーム	形式 1~4	
	QnA 互換 2C フレーム	形式 1~4	
	QnA 互換 3C フレーム	形式 1~4	
	QnA 互換 4C フレーム	形式 1~5	
Ethernet	A 互換 1E フレーム	—	
	QnA 互換 3E フレーム	—	
	4E フレーム	—	

過去の事例で、Ethernet (TCP/IP) での通信は、QnA 互換 3E フレームを使用している。
ターゲットPLCがiQRで、ラベル名によるデバイス読み込み・書き込み (SLMP通信) を行う場合、QnA 互換 3E フレームまたは 4E フレームを指定すること。

2. API 関数仕様

MELSEC通信APIは、以下の種類に分類される。

- (1) APIの初期化／解放
- (2) 通信回線のオープン、接続、クローズ
- (3) MELSECとの通信
- (4) 通信ログ

関数仕様の説明は、C++表記で行う。

2-1. APIの初期化／解放

関数名	関数処理内容	備考
MELSEC_GetVer	バージョンの取得	
MELSEC_Initialize	APIの初期化	
MELSEC_Release	APIの解放	
MELSEC_ReleaseAll	APIの解放（全インスタンス）	

MELSEC_GetVer

MELSEC通信APIのDLLバージョンを取得する。

【関数仕様】

LONG MELSEC_GetVer (TCHAR* ver);

var	バージョン番号格納バッファポインタ 16文字以上のバッファを準備すること
戻り値	バージョン番号文字数

【関数使用例】

```
TCHAR    ver[16];  
LONG     cnt;  
  
cnt = MELSEC_GetVer (ver);
```


MELSEC_Initialize

MELSEC通信APIを初期化する。MELSEC通信APIを使用する前に、必ず本APIをコールしなければならない。複数PLCとの通信が必要な場合、複数回本APIをコールして、インスタンス番号を獲得すること。

【関数仕様】

```
MELSEC_INSTANCE MELSEC_Initialize(MELSEC_PARM* parm, VOID* vptr);
```

parm	初期化パラメータポインタ
vptr	汎用ポインタ ログ書き込みコールバック時に本ポインタ値が通知されます。 本APIを管理するクラスポインタ等に使用する。 必要なければNULLを指定する。
戻り値	初期化されたインスタンス番号 =0~99：初期化正常 負の値は、初期化に失敗したことを表す。

戻り値が負の値の場合、以下の初期化不良を表す。

```
MELSEC_INSTANCE_ILLEGAL_1(-1) // インスタンス番号獲得失敗 (パラメータ不正：CPU 種類)
MELSEC_INSTANCE_ILLEGAL_2(-2) // インスタンス番号獲得失敗 (パラメータ不正：伝送フレーム)
MELSEC_INSTANCE_ILLEGAL_3(-3) // インスタンス番号獲得失敗 (パラメータ不正：通信形式)
MELSEC_INSTANCE_ILLEGAL_4(-4) // インスタンス番号獲得失敗 (パラメータ不正：伝送コード)
MELSEC_INSTANCE_ILLEGAL_5(-5) // インスタンス番号獲得失敗 (パラメータ不正：通信リトライ回数)
MELSEC_INSTANCE_ILLEGAL_6(-6) // インスタンス番号獲得失敗 (パラメータ不正：サムチェック有無)
MELSEC_INSTANCE_ILLEGAL_7(-7) // インスタンス番号獲得失敗 (インスタンス空き無し)
```

【パラメータ構造体】

```

typedef struct {
    WORD    CpuType;          // PLC CPU 種類
                                //  =MELSEC_CPU_A:   ACPU
                                //  =MELSEC_CPU_AnA: AnACPU
                                //  =MELSEC_CPU_QnA: QnACPU
                                //  =MELSEC_CPU_Qn:  QnCPU
                                //  =MELSEC_CPU_iQR: iQRCPU

    WORD    FlameType;        // 伝送フレーム型式
                                //  =MELSEC_FLAME_1C: A 互換 1C フレーム
                                //  =MELSEC_FLAME_2C: QnA 互換 2C フレーム
                                //  =MELSEC_FLAME_3C: QnA 互換 3C フレーム
                                //  =MELSEC_FLAME_4C: QnA 互換 4C フレーム
                                //  =MELSEC_FLAME_1E: A 互換 1E フレーム
                                //  =MELSEC_FLAME_3E: QnA 互換 3E フレーム
                                //  =MELSEC_FLAME_4E: 4E フレーム

    WORD    Protocol;         // 通信形式
                                //  =MELSEC_PRTCL0: 形式無し
                                //  =MELSEC_PRTCL1: 形式 1
                                //  =MELSEC_PRTCL2: 形式 2
                                //  =MELSEC_PRTCL3: 形式 3
                                //  =MELSEC_PRTCL4: 形式 4
                                //  =MELSEC_PRTCL5: 形式 5 (QnA 互換 4C フレーム時のみ有効)

    WORD    Code;             // 伝送コード形式
                                //  =MELSEC_CODE_ASCII: ASCII コード
                                //  =MELSEC_CODE_BINARY: BINARY コード

    WORD    Retry;            // 通信リトライ回数 (通信失敗時)
                                //  =0:   初回通信のみ
                                //  =1~MELSEC_RETRY_MAX: リトライ回数

    DWORD   RecvTimeOut;      // 受信タイムアウト時間[msec]
    DWORD   SendTimeOut;      // 送信タイムアウト時間[msec]
    DWORD   RetryWaitTime;    // 通信リトライ時の再送信待ち時間[msec]
    BOOL    SumCheck;         // サムチェック有無
                                //  =FALSE: 無し
                                //  =TRUE: 有り

    MELSEC_CBF_WRITE_LOG CBF_WriteLog; // 通信ログ書き込みコールバック関数
    LONG    ErrBase;          // エラーベース値
} MELSEC_PARM;

```

【パラメータ説明】

メンバ変数名	CpuType	設定内容	PLC CPU 種類
説明	通信相手のPLC種類を指定する。 =MELSEC_CPU_A: ACPU =MELSEC_CPU_AnA: AnACPU =MELSEC_CPU_QnA: QnACPU =MELSEC_CPU_Qn: QnCPU =MELSEC_CPU_iQR: iQRCPU		

メンバ変数名	FlameType	設定内容	伝送フレーム型式
説明	通信フレーム形式を指定する。MELSEC_FLAME_1C~4C はシリアル通信、MELSEC_FLAME_1E~4E は Ethernet 通信で使用する。 =MELSEC_FLAME_1C: A 互換 1C フレーム =MELSEC_FLAME_2C: QnA 互換 2C フレーム =MELSEC_FLAME_3C: QnA 互換 3C フレーム =MELSEC_FLAME_4C: QnA 互換 4C フレーム =MELSEC_FLAME_1E: A 互換 1E フレーム =MELSEC_FLAME_3E: QnA 互換 3E フレーム =MELSEC_FLAME_4E: 4E フレーム		

メンバ変数名	Protocol	設定内容	通信形式
説明	通信形式を指定する。伝送フレームがMELSEC_FLAME_1C~4C の場合、形式1~5を指定する。 伝送フレームがMELSEC_FLAME_1E~4E の場合は、形式無しを指定する。 =MELSEC_PRTCL0: 形式無し (MELSEC_FLAME_1E~4E の場合) =MELSEC_PRTCL1: 形式1 =MELSEC_PRTCL2: 形式2 =MELSEC_PRTCL3: 形式3 =MELSEC_PRTCL4: 形式4 =MELSEC_PRTCL5: 形式5 (QnA 互換 4C フレーム時のみ有効)		

メンバ変数名	Code	設定内容	伝送コード形式
説明	伝送コードを指定する。 =MELSEC_CODE_ASCII: ASCII コード =MELSEC_CODE_BINARY: BINARY コード		

メンバ変数名	Retry	設定内容	通信リトライ回数
説明	通信失敗時のリトライ回数（初回通信は、含まない）を指定する。 =0: 初回通信のみ =1～MELSEC_RETRY_MAX: リトライ回数 通信リトライ回数は、最大 MELSEC_RETRY_MAX (3) までとする。		

メンバ変数名	RecvTimeOut	設定内容	受信タイムアウト時間
説明	MELSECへコマンドを送信してから、レスポンス受信するまでのタイムアウト時間（単位：msec）を指定する。 注意）伝送フレームがMELSEC_FLAME_2C～4Cのシリアル通信の場合、大量の通信バイト数が発生する為に、十分な値（10 秒程度）を指定すること。		

メンバ変数名	SendTimeOut	設定内容	送信タイムアウト時間
説明	MELSECへコマンドを送信してから、送信完了するまでのタイムアウト時間（単位：msec）を指定する。 注意）伝送フレームがMELSEC_FLAME_2C～4Cのシリアル通信の場合、大量の通信バイト数が発生する為に、十分な値（10 秒程度）を指定すること。		

メンバ変数名	RetryWaitTime	設定内容	通信リトライ時の再送信待ち時間
説明	通信失敗時、リトライ回数が指定されている場合、通信リトライの待ち時間（単位：msec）を指定する。		

メンバ変数名	SumCheck	設定内容	サムチェック有無
説明	伝送フレーム形式がMELSEC_FLAME_1C～4Cの場合、サムチェック有無を指定する。 注）MELSEC側計算機リンクユニット（QJ71C24N-R2 等）の通信設定と一致させること。		

メンバ変数名	CBF_WriteLog	設定内容	通信ログ関数（コールバック）
説明	MELSEC通信APIからの通信ログ書き込み関数（コールバック関数）を指定する。 各通信API関数の通信ログ書き込みが指示されている場合のみコールバックされる。 コールバック関数パラメータ仕様は、MELSEC_CBF_WRITE_LOG を参照すること。		

【関数使用例】

<通信条件>

通信媒体： シリアル通信
 PLC機種： MELSEC_CPU_QnA // QnACPU
 伝送フレーム： MELSEC_FLAME_1C // A互換1Cフレーム
 通信形式： MELSEC_PRTCL1 // 形式1
 伝送コード型式： MELSEC_CODE_ASCII // 伝文コード形式(ASCII)
 通信リトライ回数： MELSEC_RETRY_MAX // 3回
 受信タイムアウト時間： 3秒
 送信タイムアウト時間： 3秒
 通信リトライ時の再送信待ち時間： 500msec
 サムチェック有無： 有り

```

/*****
MELSEC 通信 API からのコールバック関数
*****/
// 通信ログ書き込み
// MELSEC_DATA_BYTE = 8192
static void CALLBACK CBF_WriteLog(MELSEC_INSTANCE inst_no,
    CHAR* header_text, LONG blng, BYTE* data, VOID* vptr) {
    CHAR* hexdump;
    LONG hexdump_lng = (62 + 2) + ((73 + 2) * (MELSEC_DATA_BYTE / 16 + 1));
    hexdump = new TCHAR[hexdump_lng];

    // 通信データ HEX ダンプデータへ変換
    memset(hexdump, 0, hexdump_lng);
    if (blng > 0) {
        SYSLOG_MakeByteHexDump(m_syslog_inst, blng, data, hexdump_lng, hexdump);
    }
    SYSLOG_Write(m_syslog_inst, SYSLOG_COM, header_text, hexdump);

    delete hexdump;
}
  
```

コールバック関数については、次ページ参照のこと。

```

/*****
MELSEC 通信 API からのコールバック関数
*****/
static void CALLBACK CBF_WriteLog(MELSEC_INSTANCE inst_no,
    CHAR* header_text, LONG bIn, BYTE* data, VOID* vptr) {

    CHAR*    hexdump = NULL;

    if (bIn) {
        // HEX ダンプバッファの確保 (タイトル行 + HEX ダンプ行 + NULL)
        LONG    hexdump_lng = (62 + 2) + ((73 + 2) * (bIn / 16 + 1));
        hexdump = new TCHAR[hexdump_lng];
        // 通信データ HEX ダンプデータへ変換
        ::memset(hexdump, 0, hexdump_lng);
        SYSLOG_MakeByteHexDump(m_syslog_inst, bIn, data, hexdump_lng, hexdump);
    }
    // 通信ログの書込み
    SYSLOG_Write(m_syslog_inst, SYSLOG_COM, header_text, hexdump);

    // HEX ダンプバッファの解放
    if (hexdump) delete hexdump;
}

```

SYSLOG_MakeByteHexDump() を実行すると、下記の内容が編集されバッファに格納される。

hexdump 格納例

□□□ADDR +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F ASCII	← 62 + 2 (CR, LF)
□□□00000 02 30 30 46 46 51 57 30 44 30 30 30 34 30 30 30 .00FFQWOD0004000	← 73 + 2 (CR, LF)
□□□00010 42 30 31 30 31 30 33 30 30 30 33 35 30 30 30 32 B010103000350002	
□□□00020 32 32 34 33 38 30 30 30 30 31 32 30 36 30 31 2243800000120601	
□□□00030 30 30 30 31 30 33 38 30 30 30 30 30 31 03 31 41 0001038000001.1A	
0x00	

(□は、スペース文字を表す。)

```
MELSEC_INSTANCE    MELSEC_inst;
MELSEC_PARM         parm;
MELSEC_ERR          err;

// パラメータセット
::memset(&parm, 0, sizeof(parm));
parm.MELSECuType    = MELSEC_CPU_QnA;    // PLC CPU 種類
parm.FlameType      = MELSEC_FLAME_1C;    // 伝送フレーム型式
parm.Protocol       = MELSEC_PRTCL1;     // 通信形式
parm.Code           = MELSEC_CODE_ASCII; // 伝送コード形式
parm.Retry          = MELSEC_RETRY_MAX;   // 通信リトライ回数 (通信失敗時)
parm.RecvTimeOut    = 3000;               // 受信タイムアウト時間[msec]
parm.SendTimeOut    = 3000;               // 送信タイムアウト時間[msec]
parm.RetryWaitTime  = 500;                // 通信リトライ時の再送信待ち時間[msec]
parm.SumCheck       = TRUE;               // サムチェック有無
parm.CBF_WriteLog   = CBF_WriteLog;      // コールバック関数

// API 初期化
MELSEC_inst = MELSEC_Initialize(&parm, NULL);
if (MELSEC_inst >= 0) {
    // 通信処理

    // システム終了時、API の解放
    err = MELSEC_Release(MELSEC_inst);
}
```


MELSEC_Release

MELSEC通信APIの解放を行う。MELSEC_Initialize 実施後、アプリケーションを終了する前には、必ず本APIをコールしなければならない。

【関数仕様】

MELSEC_ERR	MELSEC_Release(MELSEC_INSTANCE inst_no);
inst_no	初期化したインスタンス番号
戻り値	エラー番号 (=0 : 正常完了、≠0 : 異常完了)

【関数使用例】

```
MELSEC_INSTANCE  MELSEC_inst;
MELSEC_PARM       MELSEC_parm;
MELSEC_ERR        err;

// 初期化パラメータセット
;
// API 初期化
MELSEC_inst = MELSEC_Initialize(&MELSEC_parm, NULL);
if (MELSEC_inst >= 0) {
    // 通信処理

// システム終了時、API の解放
err = MELSEC_Release(MELSEC_inst);
}
```

MELSEC_ReleaseAll

MELSEC通信APIのすべてを解放する。MELSEC_Initialize 実施後、アプリケーションを終了する前には、必ず MELSEC_Release または本API をコールしなければならない。

【関数仕様】

MELSEC_ERR MELSEC_ReleaseAll(void);

戻り値 エラー番号 (=0 : 正常完了、≠0 : 異常完了)

【関数使用例】

```
enum {
    PLC1 = 0,      // PLC1
    PLC2,          // PLC2
    PLC_CNT
};
MELSEC_INSTANCE  MELSEC_inst[PLC_CNT];
MELSEC_PARM      MELSEC_parm[PLC_CNT];
MELSEC_ERR       err;

// 初期化/パラメータセット
;

// API 初期化
MELSEC_inst[PLC1] = MELSEC_Initialize(&MELSEC_parm[PLC1], NULL);
MELSEC_inst[PLC2] = MELSEC_Initialize(&MELSEC_parm[PLC2], NULL);

if ((MELSEC_inst[PLC1] >= 0) && (MELSEC_inst[PLC2] >= 0)) {
    // 通信処理

    // システム終了時、PLC1 PLC2 のAPI 解放
    err = MELSEC_ReleaseAll();
}
```

2-2. 通信回線のオープン、接続、クローズ

関数名	関数処理内容	備考
MELSEC_Open	シリアル通信ポートのオープン	
MELSEC_Connect	Ethernet 通信回線の接続	
MELSEC_Close	シリアルまたはEthernet 通信回線のクローズ	
MELSEC_IsOpen	通信回線のオープンまたは接続状態の獲得	

【パラメータ構造体】

```
typedef struct {
    TCHAR    ComPort[16];    // COM ポート番号
                                //   ="COM1"~"COMn"
    DWORD    BaudRate;       // 通信ボーレート
                                //   =CBR_300:   300[bps]
                                //   =CBR_600:   600[bps]
                                //   =CBR_1200:  1200[bps]
                                //   =CBR_4800:  4800[bps]
                                //   =CBR_9600:  9600[bps]
                                //   =CBR_19200: 19200[bps]
    BYTE     ByteSize;       // バイトサイズ
                                //   =7: 7 ビット
                                //   =8: 8 ビット
    BYTE     Parity;         // パリティ
                                //   =NOPARITY:   パリティなし
                                //   =ODDPARITY:   奇数パリティ
                                //   =EVENPARITY:  偶数パリティ
    BYTE     StopBits;       // ストップビット
                                //   =ONESTOPBIT:  1 ストップビット
                                //   =TWOSTOPBITS: 2 ストップビット
    BYTE     yobi;           //
    DWORD    RtsControl;     // RTS 制御
                                //   =RTS_CONTROL_*
    DWORD    ByteTimeOut;    // 受信バイト間タイムアウト[msec]
} MELSEC_OPEN_PARM;
```

【パラメータ説明】

メンバ変数名	ComPort	設定内容	COM ポート番号
説明	シリアル回線ポート番号を指定する。 例) =“COM1”: COM1 ポート番号 =“COM2”: COM2 ポート番号		

メンバ変数名	BaudRate	設定内容	通信ボーレート
説明	通信速度（ボーレート）を指定する。指定する値は、WinBase.h 内に定義されている define 値を指定すること。 例) =CBR_300: 300[bps] =CBR_600: 600[bps] =CBR_1200: 1200[bps] =CBR_4800: 4800[bps] =CBR_9600: 9600[bps] =CBR_19200: 19200[bps]		

メンバ変数名	ByteSize	設定内容	バイトサイズ
説明	通信バイトサイズを指定する。指定する値は、7 または 8 を指定すること。 例) =7: 7 ビット =8: 8 ビット		

メンバ変数名	Parity	設定内容	パリティ
説明	通信パリティを指定する。指定する値は、WinBase.h 内に定義されている define 値を指定すること。 例) =NOPARITY: パリティなし =ODDPARITY: 奇数パリティ =EVENPARITY: 偶数パリティ		

メンバ変数名	StopBits	設定内容	ストップビット
説明	通信ストップビット長を指定する。指定する値は、WinBase.h 内に定義されている define 値を指定すること。 例) =ONESTOPBIT: 1 ストップビット =TWOSTOPBITS: 2 ストップビット		

メンバ変数名	RtsControl	設定内容	RTS 制御
説明	シリアル回線の RS 制御を指定する。指定する値は、WinBase.h 内に定義されている define 値を指定すること。 例) RTS_CONTROL_DISABLE: RS 制御無し RTS_CONTROL_ENABLE: RS 制御有り		

メンバ変数名	ByteTimeOut	設定内容	受信バイト間タイムアウト時間
説明	通信バイト間タイムアウト時間を指定する。データ受信時、本値以上の時間（単位：msec）が経過した場合、受信完了とする。		

【関数使用例】

<回線オープン条件>

通信媒体:	シリアル通信
COM ポート番号:	COM1
通信ボーレート:	9600bps
バイトサイズ:	8 ビット
パリティ:	偶数パリティ
ストップビット長:	1 ビット
RTS 制御:	有効
受信バイト間タイムアウト時間:	20msec
通信ログ書込み有無	行う

```

MELSEC_INSTANCE    MELSEC_inst;
MELSEC_OPEN_PARM   open_parm;
MELSEC_ERR         err;

// 初期化パラメータセット
;

// オープンパラメータセット
::memset(&open_parm, 0, sizeof(open_parm));
// 通信ポート番号
::strcpy(open_parm.ComPort, "COM1");
open_parm.BaudRate   = CBR_9600;           // 通信ボーレート
open_parm.ByteSize   = 8;                 // バイトサイズ
open_parm.Parity      = EVENPARITY;        // パリティ
open_parm.StopBits    = ONESTOPBIT;       // ストップビット長
open_parm.RtsControl  = RTS_CONTROL_ENABLE; // RTS 制御
open_parm.ByteTimeOut = 20;               // 受信バイト間タイムアウト時間[msec]

// シリアル通信回線オープン
err = MELSEC_Open(MELSEC_inst, &open_parm, TRUE);
if (err == 0) {
    // 通信処理

    // システム終了時、シリアル通信回線クローズ
    err = MELSEC_Close(MELSEC_inst, TRUE);
}

```


MELSEC_Connect

Ethernet 回線の接続を行う。伝送フレームが MELSEC_FLAME_1E~4E の場合、本関数を使用する。

【関数仕様】

```
MELSEC_ERR      MELSEC_Connect (MELSEC_INSTANCE inst_no, MELSEC_CONNECT_PARM* parm,  
                                  BOOL log_enable);
```

inst_no	初期化したインスタンス番号
parm	回線接続パラメータポインタ
log_enable	通信ログ書込み有無（コールバック） =TRUE: 行う =FALSE: 行わない

戻り値	エラー番号（=0：正常完了、≠0：異常完了）
-----	------------------------

【パラメータ構造体】

```
// プロトコル
```

```
enum {
```

```
    MELSEC_TCP_IP = 0,          // TCP/IP
```

```
    MELSEC_UDP_IP              // UDP/IP
```

```
};
```

```
typedef struct {
```

```
    TCHAR    IpAddr[16];        // PLC IP アドレス  
                                //    ="192.168.2.101" (設定例)
```

```
    WORD     PortNo;            // PLC ポート番号
```

```
    WORD     LocalPortNo;       // 自局ポート番号 (UDP/IP の場合、設定)
```

```
    WORD     Protocol;          // 通信プロトコル
```

```
                                //    =MELSEC_TCP_IP: TCP/IP
```

```
                                //    =MELSEC_UDP_IP: UDP/IP
```

```
    DWORD    ConTimeOut;        // 回線接続監視時間[msec]
```

```
} MELSEC_CONNECT_PARM;
```

【パラメータ説明】

メンバ変数名	IpAddr	設定内容	PLC IP アドレス
説明	PLC回線接続のIPアドレスを指定する。 例) ="192.168.2.250"		

メンバ変数名	PortNo	設定内容	PLC ポート番号
説明	PLC回線接続のポート番号を指定する。 例) =4096		

メンバ変数名	LocalPortNo	設定内容	自局ポート番号
説明	UDP/IPで通信する場合、自局ポート番号を指定する。 例) =4095		

メンバ変数名	Protocol	設定内容	通信プロトコル
説明	Ethernet 通信プロトコルを指定する。 例) =MELSEC_TCP_IP: TCP/IP =MELSEC_UDP_IP: UDP/IP		

メンバ変数名	ConTimeOut	設定内容	回線接続監視時間
説明	回線接続監視時間（単位：msec）を指定する。		

【関数使用例】

<Ethernet 回線接続条件>

通信媒体 :	Ethernet
PLC IP アドレス :	192.168.2.250
PLC ポート番号 :	4096
通信プロトコル :	TCP/IP
回線接続監視時間 :	3 秒
通信ログ書き込み有無	行う

```

MELSEC_INSTANCE      MELSEC_inst;
MELSEC_CONNEC_PARM   connect_parm;
MELSEC_ERR            err;

// 初期化パラメータセット
;

// 回線接続パラメータセット
::memset(&connect_parm, 0, sizeof(MELSEC_connect_parm));
// IP アドレス
::strcpy(connect_parm.IpAddr, "192.168.2.250");
connect_parm.PortNo    = 4096;           // PLC ポート番号
connect_parm.Protocol  = MELSEC_TCP_IP;  // 通信プロトコル (TCP/IP)
connect_parm.ConTimeOut = 3000;          // 回線接続監視時間[msec]

// Ethernet 通信回線接続
err = MELSEC_Connect(MELSEC_inst, &connect_parm, TRUE);
if (err == 0) {
    // 通信処理

    // システム終了時、シリアル通信回線クローズ
    err = MELSEC_Close(MELSEC_inst, TRUE);
}

```

MELSEC_Close

通信回線のクローズ（切断）を行う。

【関数仕様】

```
MELSEC_ERR      MELSEC_Close(MELSEC_INSTANCE inst_no, BOOL log_enable);
```

inst_no	初期化したインスタンス番号
log_enable	通信ログ書込み有無（コールバック） =TRUE: 行う =FALSE: 行わない
戻り値	エラー番号（=0: 正常完了、≠0: 異常完了）

MELSEC_IsOpen

通信回線のオープン（接続）状態を獲得する。

【関数仕様】

```
BOOL    MELSEC_IsOpen(MELSEC_INSTANCE inst_no);
```

inst_no	初期化したインスタンス番号
---------	---------------

戻り値	処理結果
	=TRUE : 回線オープン（接続）中
	=FALSE : 回線未オープン（切断）中

【エラー番号】

ERR_MELSEC_0001	// 初期化パラメータ内容不正
ERR_MELSEC_0002	// 初期化失敗（空きのインスタンス不足）
ERR_MELSEC_0003	// API 処理失敗（インスタンス未初期化）
ERR_MELSEC_0004	// API 解放失敗（インスタンス番号不正）
ERR_MELSEC_0005	// API 解放失敗（インスタンス未初期化）
ERR_MELSEC_0010	// シリアル回線オープン異常（初期化伝送フレーム設定異常）
ERR_MELSEC_0011	// シリアル回線オープン異常（既に回線オープン中）
ERR_MELSEC_0012	// シリアル回線オープン異常（回線オープンに失敗）
ERR_MELSEC_0015	// シリアル回線パージ失敗
ERR_MELSEC_0021	// シリアル回線データ送信異常（回線未オープン状態）
ERR_MELSEC_0022	// シリアル回線データ送信異常（0バイト送信完了）
ERR_MELSEC_0023	// シリアル回線データ送信異常（送信ペンディングに失敗）
ERR_MELSEC_0024	// シリアル回線データ送信異常（送信タイムアウト）
ERR_MELSEC_0031	// シリアル回線データ受信異常（回線未オープン状態）
ERR_MELSEC_0032	// シリアル回線データ受信異常（イベント登録に失敗）
ERR_MELSEC_0033	// シリアル回線データ受信異常
ERR_MELSEC_0034	// シリアル回線データ受信異常（受信タイムアウト）
ERR_MELSEC_0035	// シリアル回線データ受信異常（エラー情報の取得に失敗）
ERR_MELSEC_0036	// シリアル回線データ受信異常（通信エラーが発生）
ERR_MELSEC_0037	// シリアル回線データ受信異常（受信データ読込に失敗）
ERR_MELSEC_0038	// シリアル回線データ受信異常（受信データ読込に失敗）
ERR_MELSEC_0039	// シリアル回線データ受信異常（受信データ読込に失敗）
ERR_MELSEC_0040	// Ethernet 回線接続異常（初期化伝送フレーム設定異常）
ERR_MELSEC_0041	// Ethernet 回線接続異常（既に回線オープン中）
ERR_MELSEC_0042	// Ethernet 回線接続異常（ソケット作成に失敗）
ERR_MELSEC_0043	// Ethernet 回線接続異常（イベント登録に失敗）
ERR_MELSEC_0044	// Ethernet 回線接続異常（イベント選択に失敗）
ERR_MELSEC_0045	// Ethernet 回線接続異常（接続監視タイムアウト）
ERR_MELSEC_0046	// Ethernet 回線接続異常（イベント種類抽出に失敗）
ERR_MELSEC_0047	// Ethernet 回線接続異常（イベント結果不正）
ERR_MELSEC_0051	// Ethernet 回線データ送信異常（回線未接続状態）
ERR_MELSEC_0052	// Ethernet 回線データ送信異常（送信に失敗）
ERR_MELSEC_0061	// Ethernet 回線データ受信異常（回線未接続状態）
ERR_MELSEC_0062	// Ethernet 回線データ受信異常（回線切断検出）
ERR_MELSEC_0063	// Ethernet 回線データ受信異常（受信タイムアウト）
ERR_MELSEC_0090	// 回線クロース異常（初期化伝送フレーム設定異常）
ERR_MELSEC_0091	// シリアル回線クロース異常（回線未オープン状態）
ERR_MELSEC_0092	// Ethernet 回線切断異常（回線未接続状態）
ERR_MELSEC_0100	// コマンド編集異常（該当フレーム未サポート）
ERR_MELSEC_0101	// コマンド編集異常（指定点数オーバー）

ERR_MELSEC_0200 // レスポンス受信に失敗
ERR_MELSEC_0201 // レスポンス内容異常 (先頭コード不正)
ERR_MELSEC_0202 // レスポンス内容異常 (サブヘッダ不正)
ERR_MELSEC_0203 // レスポンス内容異常 (ブロック番号/シリアル番号不一致)
ERR_MELSEC_0204 // レスポンス内容異常 (局番号不一致)
ERR_MELSEC_0205 // レスポンス内容異常 (PC 番号不一致)
ERR_MELSEC_0206 // レスポンス内容異常 (ETX コード不正)
ERR_MELSEC_0207 // レスポンス内容異常 (サムチェック不一致)
ERR_MELSEC_0208 // レスポンス内容異常 (ネットワーク番号不一致)
ERR_MELSEC_0209 // レスポンス内容異常 (要求先ユニット I/O 番号不一致)
ERR_MELSEC_0210 // レスポンス内容異常 (要求先ユニット局番号不一致)
ERR_MELSEC_0211 // レスポンス内容異常 (応答データ長不一致)
ERR_MELSEC_0212 // レスポンス内容異常 (フレーム識別番号不正)
ERR_MELSEC_0213 // レスポンス内容異常 (自局番号不一致)
ERR_MELSEC_0214 // レスポンス内容異常 (応答識別コード不正)
ERR_MELSEC_0215 // レスポンス内容異常 (固定値不正)
ERR_MELSEC_0300 // 通信ログエクスポート異常 (ファイルオープンに失敗)
ERR_MELSEC_0301 // 通信ログエクスポート異常 (ファイルアクセス障害発生)

// PLCからの応答エラーコード (ERR_MELSEC_1000 + PLC 応答エラーコード)
ERR_MELSEC_1000~ // レスポンス内容異常 (PLC エラー応答)
// 1000+PLC 応答エラーコード

2-3. MELSECとの通信

MELSECとの通信APIは、各関数で通信パラメータを指定する必要がある。
デフォルトのパラメータ値を獲得するには、MELSEC_GetComParm を使用する。

関数名	関数処理内容	備考
MELSEC_DevStr	デバイス番号文字列からデバイス番号へ変換	
MELSEC_DevStrCs	デバイス番号文字列からデバイス番号へ変換	
MELSEC_Dev	デバイス種類、デバイス番号からデバイス番号へ変換	
MELSEC_DevCs	デバイス種類、デバイス番号からデバイス番号へ変換	
MELSEC_GetDevInfo	デバイス情報の獲得	
MELSEC_GetComParm	通信パラメータ（現在設定値）の読み込み	
MELSEC_ReadBits	ビットデバイスの一括読み込み	
MELSEC_ReadWords	ワードデバイスの一括読み込み	
MELSEC_WriteBits	ビットデバイスの一括書き込み	
MELSEC_WriteWords	ワードデバイスの一括書き込み	
MELSEC_RemoteCpu	CPU に対してリモート操作	
MELSEC_GetCpuType	CPU 形名読み込み	
MELSEC_LoopbackTest	ループバックテスト	
MELSEC_ReadArrayLabel	ラベル指定のデバイス配列一括読み込み	iQR CPU のみサポート
MELSEC_WriteArrayLabel	ラベル指定のデバイス配列一括書き込み	iQR CPU のみサポート

MELSEC_DevStr

デバイス番号文字列からデバイス番号構造体値を獲得する。

【関数仕様】

```
MELSEC_DEV      MELSEC_DevStr (LPCTSTR dev_str);
```

dev_str	デバイス番号文字列ポインタ
---------	---------------

戻り値	デバイス番号構造体値
-----	------------

【デバイス番号構造体】

```
typedef struct {  
    DWORD    No;           // デバイス番号  
    WORD     Code;        // デバイス種類  
} MELSEC_DEV;
```

MELSEC_DevStrCs

デバイス番号文字列からデバイス番号構造体値を獲得する。
本関数は、主にC#のアプリケーションで使用する。

【関数仕様】

```
BOOL    MELSEC_DevStr2(LPCTSTR dev_str, MELSEC_DEV *dev);
```

dev_str	デバイス番号文字列ポインタ
dev	デバイス番号格納ポインタ

戻り値	処理結果
	=TRUE: 正常
	=FALSE: 失敗

【デバイス番号構造体】

```
typedef struct {  
    DWORD    No;           // デバイス番号  
    WORD     Code;        // デバイス種類  
} MELSEC_DEV;
```

// デバイス種類

#define MELSEC_DEV_M	0x90	// 内部リレー (A, QnA 特殊リレー含む)
#define MELSEC_DEV_SM	0x91	// 特殊リレー
#define MELSEC_DEV_D	0xA8	// データレジスタ (A, QnA 特殊レジスタ含む)
#define MELSEC_DEV_SD	0xA9	// 特殊レジスタ
#define MELSEC_DEV_X	0x9C	// 入力リレー
#define MELSEC_DEV_Y	0x9D	// 出力リレー
#define MELSEC_DEV_L	0x92	// ラッチリレー
#define MELSEC_DEV_F	0x93	// アナンシェータ
#define MELSEC_DEV_V	0x94	// エッジリレー
#define MELSEC_DEV_B	0xA0	// リンクリレー
#define MELSEC_DEV_W	0xB4	// リンクレジスタ
#define MELSEC_DEV_TS	0xC1	// タイマ接点
#define MELSEC_DEV_TC	0xC0	// タイマコイル
#define MELSEC_DEV_TN	0xC2	// タイマ現在値
#define MELSEC_DEV_SS	0xC7	// 積算タイマ接点
#define MELSEC_DEV_SC	0xC6	// 積算タイマコイル
#define MELSEC_DEV_SN	0xC8	// 積算タイマ現在値
#define MELSEC_DEV_CS	0xC4	// カウンタ接点
#define MELSEC_DEV_CC	0xC3	// カウンタコイル
#define MELSEC_DEV_CN	0xC5	// カウンタ現在値
#define MELSEC_DEV_SB	0xA1	// リンク特殊リレー
#define MELSEC_DEV_SW	0xB5	// リンク特殊レジスタ
#define MELSEC_DEV_S	0x98	// ステップリレー
#define MELSEC_DEV_DX	0xA2	// ダイレクト入力
#define MELSEC_DEV_DY	0xA3	// ダイレクト出力
#define MELSEC_DEV_Z	0xCC	// インデックスレジスタ
#define MELSEC_DEV_R	0xAF	// ファイルレジスタ
#define MELSEC_DEV_ZR	0xB0	// ファイルレジスタ

【関数使用例】

```
MELSEC_DEV      dev;  
WORD             dev_data[256];  
MELSEC_ERR       err;
```

```
dev = MELSEC_DevStr("D1234");
```

```
// dev には、下記の値がセットされる。
```

```
dev. No          1234  
dev. Code        MELSEC_DEV_D (0xA8)
```

```
// ビットデバイスの読み込み (M123 から 10 ビット読み込み)
```

```
err = MELSEC_ReadBits(MELSEC_inst, &com_parm, MELSEC_DevStr("M123"), 10, dev_data, TRUE);
```

MELSEC_Dev

デバイス種類、デバイス番号からデバイス番号構造体値を獲得する。

【関数仕様】

```
MELSEC_DEV      MELSEC_Dev (WORD dev_code, WORD dev_no);
```

dev_code	デバイス種類
dev_no	デバイス番号（番号部分のみ）
戻り値	デバイス番号構造体値

MELSEC_DevCs

デバイス種類、デバイス番号からデバイス番号構造体値を獲得する。
本関数は、主にC#のアプリケーションで使用する。

【関数仕様】

```
VOID      MELSEC_DevCs (WORD dev_code, WORD dev_no, MELSEC_DEV* dev);
```

dev_code	デバイス種類
dev_no	デバイス番号（番号部分のみ）
dev	デバイス番号格納ポインタ
戻り値	無し

【関数使用例】

```
MELSEC_DEV      MELSEC_dev;  
WORD             dev_data[256];  
BOOL             rc;  
  
MELSEC_dev = MELSEC_Dev(MELSEC_DEV_M, 123);  
// MELSEC_dev には、下記の値がセットされる。  
MELSEC_dev.No      123  
MELSEC_dev.Code    MELSEC_DEV_M(0x90)  
  
// ビットデバイスの読み込み (M123 から 10 ビット読み込み)  
rc = MELSEC_ReadBits(MELSEC_inst, &com_parm, MELSEC_Dev(MELSEC_DEV_M, 123), 10, dev_data, TRUE);
```

MELSEC_GetDevInfo

デバイス情報を獲得する。

【関数仕様】

```
MELSEC_ERR MELSEC_GetDevInfo(MELSEC_DEV MELSEC_dev, MELSEC_DEV_INFO* dev_info);
```

MELSEC_dev	デバイス番号構造体値
dev_info	デバイス情報格納ポインタ
戻り値	エラー番号 (=0 : 正常完了、≠0 : 異常完了)

【デバイス情報構造体】

```
// デバイス番号形態
enum {
    MELSEC_DEV_NUM_DEC = 0,    // デバイス番号 10 進数
    MELSEC_DEV_NUM_HEX        // デバイス番号 16 進数
};
// デバイス種類
enum {
    MELSEC_DEV_BIT = 0,        // ビットデバイス
    MELSEC_DEV_WORD            // ワードデバイス
};

typedef struct {
    WORD    Code;                // デバイスコード
                                   //      =MELSEC_DEV_xx
    TCHAR    DevStr[4];          // デバイスコード (文字列)
                                   //      例) "M"
    WORD    NumType;            // デバイス番号数値種類
                                   //      =MELSEC_DEV_NUM_DEC: 10 進数
                                   //      =MELSEC_DEV_NUM_HEX: 16 進数
    WORD    DevType;            // デバイス種類
                                   //      =MELSEC_DEV_BIT:   ビットデバイス
                                   //      =MELSEC_DEV_WORD:  ワードデバイス
} MELSEC_DEV_INFO;
```


【関数使用例】

```
MELSEC_DEV_INFO    dev_info;
MELSEC_ERR          err;

err = MELSEC_GetDevInfo(MELSEC_DevStr("M123"), &dev_info);
// デバイス情報構造体には、下記の値がセットされる
dev_info.Code        MELSEC_DEV_M
dev_info.DevStr       "M" (文字列の先頭ポインタ)
dev_info.NumType      MELSEC_DEV_NUM_DEC (デバイス番号 10 進数表記)
dev_info.DevType      MELSEC_DEV_BIT (ビットデバイス)
```

MELSEC_GetComParm

通信パラメータ（現在設定値）を読み込む。通信パラメータは本API初期化時、デフォルト値がセットされている。各通信関数のパラメータで指定された通信パラメータは、本API内に現在値として記憶される。

【関数仕様】

MELSEC_ERR	MELSEC_GetComParm(MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm);
inst_no	初期化したインスタンス番号
parm	通信パラメータ格納ポインタ
戻り値	エラー番号（=0：正常完了、≠0：異常完了）

【関数使用例】

```
MELSEC_COM_PARAM  com_parm;
MELSEC_ERR         err;

// 通信パラメータ（現在設定値）の読み込み
err = MELSEC_GetComParm(MELSEC_inst, &com_parm);
```

【通信パラメータ構造体】

```

// ブロック番号
#define MELSEC_BLOCK_NO          0x00    // 0x00～0xFF

// 局番号
#define MELSEC_ST_NO             0x00    // 通信ユニット設定局番
                                         // 0x00: 1対1で通信する場合
                                         // 0x01～0x1F: マルチドロップで通信する場合の相手PLC局番号

// ネットワーク番号
#define MELSEC_NET_NO            0x00    // 通信ユニット搭載PLC自身のネットワーク番号
                                         // 0x01～0xEF: アクセス局のネットワーク番号

// PC番号
#define MELSEC_PC_NO             0xFF    // 通信ユニット搭載PLC自局番号
                                         // 0x00～0x40: MELSECNET上のPLC PC番号

// 自局番号
#define MELSEC_MY_NO             0x00    // 既定値

// ネットワーク番号
#define MELSEC_NET_NO            0x00    // 既定値

// 要求先ユニットI/O番号
#define MELSEC_INIT_IO_NO        0x03FF  // 通信ユニット搭載PLC自身のユニットI/O番号

// 要求先ユニット局番号
#define MELSEC_UNIT_ST_NO        0x00    // 通信ユニット搭載PLC自身のユニット局番号

// パラメータ構造体
typedef struct {
    BYTE    BlockNo;           // ブロック番号
    BYTE    StNo;              // 局番
    BYTE    PcNo;              // PC番号
    BYTE    MyNo;              // 自局番号 (2C、3C、4C フレーム時のみ有効)
    BYTE    NetNo;             // ネットワーク番号 (4C、3E、4E フレーム時のみ有効)
    BYTE    yobi1[3];          //
    WORD    UnitIoNo;          // 要求先ユニットI/O番号 (3E、4E フレーム時のみ有効)
    BYTE    UnitStNo;          // 要求先ユニット局番号 (3E、4E フレーム時のみ有効)
    BYTE    yobi2[3];          //
    WORD    WaitTime;          // 伝文ウェイト時間 (CPU監視タイマ) [msec]
                                         // =0, 10～ (10msec刻み)
} MELSEC_COM_PARM;

```

【パラメータ説明】

メンバ変数名	BlockNo	設定内容	ブロック番号
説明	伝送フレーム、通信形式が下記の場合、指定する。 MELSEC_FLAME_1C～4C、通信形式 2 MELSEC_FLAME_4E、 値は任意で、0x00～0xFF の範囲で指定する。(0x00 固定でも可)		

メンバ変数名	StNo	設定内容	局番
説明	伝送フレーム、通信形式が下記の場合、指定する。 MELSEC_FLAME_1C～4C 通信ユニット設定局番 0x00: 1 対 1 で通信する場合、MELSEC_ST_NO (0x00) を指定する。 0x01～0x1F: シリアル通信マルチドロップで通信する場合の相手 PLC 局番号		

メンバ変数名	PcNo	設定内容	PC 番号
説明	伝送フレーム、通信形式が下記の場合、指定する。 MELSEC_FLAME_1C、3C、4C MELSEC_FLAME_1E～4E 通信ユニット搭載 PLC 自局番号 0xFF: 1 対 1 で通信する場合、MELSEC_PC_NO (0xFF) を指定する。 0x00～0x40: MELSECNET 上の PLC PC 番号		

メンバ変数名	MyNo	設定内容	自局番号
説明	伝送フレーム、通信形式が下記の場合、指定する。 MELSEC_FLAME_1C、4C MELSEC_FLAME_1E～4E 通常は、MELSEC_MY_NO (0x00) を指定する。		

メンバ変数名	NetNo	設定内容	ネットワーク番号
説明	伝送フレーム、通信形式が下記の場合、指定する。 MELSEC_FLAME_4C MELSEC_FLAME_3E、4E 通常は、MELSEC_NET_NO(0x00)を指定する。		

メンバ変数名	UnitIoNo	設定内容	要求先ユニット I/O 番号
説明	伝送フレーム、通信形式が下記の場合、指定する。 MELSEC_FLAME_3E、4E 通常は、MELSEC_INIT_IO_NO(0x03FF)を指定する。		

メンバ変数名	UnitStNo	設定内容	要求先ユニット局番号
説明	伝送フレーム、通信形式が下記の場合、指定する。 MELSEC_FLAME_3E、4E 通常は、MELSEC_UNIT_ST_NO(0x00)を指定する。		

メンバ変数名	WaitTime	設定内容	伝文ウェイト時間
説明	伝送フレーム、通信形式が下記の場合、指定する。 MELSEC_FLAME_1C MELSEC_FLAME_1E～4E PLCからの応答伝文送信ウェイト時間（単位：msec）を指定する。 通常は、0を指定する。		

【パラメータ設定マトリクス表】

設定項目	伝送フレーム						
	1C	2C	3C	4C	1E	3E	4E
ブロック番号	形式 2	形式 2	形式 2	形式 2	—	—	○
局番	○	○	○	○	—	—	—
PC 番号	○	—	○	○	—	○	○
自局番号	—	○	○	○	—	—	—
ネットワーク番号	—	—	○	○	—	○	○
要求先ユニット I/O 番号	—	—	—	○	—	○	○
要求先ユニット局番号	—	—	—	○	—	○	○
伝文ウェイト時間	○	—	—	—	○	○	○

※4E 伝送フレームの場合、ブロック番号はシリアル番号として使用される。

【API 使用可否表】

API 関数名	伝送フレーム						
	1C	2C	3C	4C	1E	3E	4E
MELSEC_ReadBits	○	○	○	○	○	○	○
MELSEC_ReadWords	○	○	○	○	○	○	○
MELSEC_WriteBits	○	○	○	○	○	○	○
MELSEC_WriteWords	○	○	○	○	○	○	○
MELSEC_RemoteCpu	×	×	○	○	×	○	○
MELSEC_GetCpuType	×	×	○	○	×	○	○
MELSEC_LoopbackTest	○	×	○	○	×	○	○
MELSEC_ReadArrayLabel	×	×	×	×	×	○	○
MELSEC_WriteArrayLabel	×	×	×	×	×	○	○

MELSEC_ReadBits

ビットデバイスのON/OFF状態を読み込む。

【関数仕様】

```
MELSEC_ERR      MELSEC_ReadBits(MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm,  
                                  MELSEC_DEV dev, WORD cnt, WORD* data, BOOL log_enable);
```

inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
dev	デバイス番号構造体値
cnt	読み込み点数
data	読み込みデータ格納ポインタ
log_enable	ON/OFF状態は、下位ビットからセットされる。 通信ログ書き込み有無（コールバック） =TRUE: 行う =FALSE: 行わない
戻り値	エラー番号（=0：正常完了、≠0：異常完了）

【関数使用例】

```
WORD          dev_data[256];  
MELSEC_ERR    err;
```

```
// ビットデバイスの読み込み (M123 から 10 ビット読み込み)
```

```
err = MELSEC_ReadBits(MELSEC_inst, &com_parm, MELSEC_DevStr("M123"), 10, dev_data, TRUE);
```

読み込みデータ (dev_data[0]) 内容

ビット番号	内容	備考
0	M123 の状態	
1	M124 の状態	
2	M125 の状態	
3	M126 の状態	
4	M127 の状態	
5	M128 の状態	
6	M129 の状態	
7	M130 の状態	
8	M131 の状態	
9	M132 の状態	
10～15	すべて 0	

MELSEC_ReadWords

ワードデバイス値またはビットデバイス状態をワード単位で読み込む。

【関数仕様】

```
MELSEC_ERR      MELSEC_ReadWords(MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm,
                                   MELSEC_DEV dev, WORD cnt, WORD* data, BOOL log_enable);
```

inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
dev	デバイス番号構造体値
cnt	読み込み点数
data	読み込みデータ格納ポインタ デバイス番号がビットデバイスの場合、 ON/OFF状態は、下位ビットからセットされる。
log_enable	通信ログ書込み有無（コールバック） =TRUE: 行う =FALSE: 行わない
戻り値	エラー番号（=0：正常完了、≠0：異常完了）

【関数使用例】

```
WORD      dev_data[256];
MELSEC_ERR err;

// ワードデバイスの読み込み (D123 から 10 ワード読み込み)
err = MELSEC_ReadWords(MELSEC_inst, &com_parm, MELSEC_DevStr("D123"), 10, dev_data, TRUE);

// ビットデバイスの読み込み (M123 から 2 ワード[32 ビット]読み込み)
err = MELSEC_ReadWords(MELSEC_inst, &com_parm, MELSEC_DevStr("M123"), 2, dev_data, TRUE);
```

MELSEC_WriteBits

ビットデバイスのON/OFF状態を書き込む。

【関数仕様】

```
MELSEC_ERR      MELSEC_WriteBits(MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm,
                                   MELSEC_DEV dev, WORD cnt, WORD* data, BOOL log_enable);
```

inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
dev	デバイス番号構造体値
cnt	書き込み点数
data	書き込みデータ格納ポインタ ON/OFF状態は、下位ビットからセットすること。
log_enable	通信ログ書き込み有無（コールバック） =TRUE: 行う =FALSE: 行わない

戻り値	エラー番号（=0：正常完了、≠0：異常完了）
-----	------------------------

【関数使用例】

```
WORD      dev_data[256];
MELSEC_ERR err;
LONG      i;

// ビットデバイスの書き込み (M123 から 10 ビット ON 書き込み)
dev_data[0] = 0;
for (i = 0; i < 10; i++) {
    data[0] |= (0x0001 << i);
}
err = MELSEC_WriteBits(MELSEC_inst, &com_parm, MELSEC_DevStr("M123"), 10, dev_data, TRUE);
```

MELSEC_WriteWords

ワードデバイス値またはビットデバイス状態をワード単位で書き込む。

【関数仕様】

MELSEC_ERR	MELSEC_WriteWords(MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm, MELSEC_DEV dev, WORD cnt, WORD* data, BOOL log_enable);
inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
dev	デバイス番号構造体値
cnt	書き込み点数
data	書き込みデータ格納ポインタ デバイス番号がビットデバイスの場合、 ON/OFF状態は、下位ビットからセットすること。
log_enable	通信ログ書き込み有無（コールバック） =TRUE: 行う =FALSE: 行わない
戻り値	エラー番号（=0：正常完了、≠0：異常完了）

【関数使用例】

```
WORD          dev_data[256];
MELSEC_ERR    err;
LONG          i;

// ワードデバイスの書き込み (D123 から 10 ワード書き込み)
for (i = 0; i < 10; i++) {
    dev_data[i] = (WORD)i;
}
err = MELSEC_WriteWords(MELSEC_inst, &com_parm, MELSEC_DevStr("D123"), 10, dev_data, TRUE);

// ビットデバイスの書き込み (M123 から 2 ワード[32 ビット] ON 書き込み)
dev_data[0] = 0xFFFF;
dev_data[1] = 0xFFFF;
err = MELSEC_WriteWords(MELSEC_inst, &com_parm, MELSEC_DevStr("M123"), 2, dev_data, TRUE);
```

ビットデバイス番号は、16の倍数でなくても良い。

MELSEC_RemoteCpu

CPUに対して、リモート操作を行う。

【関数仕様】

MELSEC_ERR	MELSEC_RemoteCpu(MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm, WORD mode, BOOL log_enable);
inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
mode	リモート操作モード =MELSEC_REMOTE_STOP: STOP =MELSEC_REMOTE_RUN: RUN =MELSEC_REMOTE_PAUSE: PAUSE =MELSEC_REMOTE_CLEAR_RUN: RUN (データメモリクリア+RUN) =MELSEC_REMOTE_LATCH_CLEAR: ラッチクリア =MELSEC_REMOTE_RESET: RESET
log_enable	通信ログ書込み有無 (コールバック) =TRUE: 行う =FALSE: 行わない
戻り値	エラー番号 (=0 : 正常完了、≠0 : 異常完了)

【関数使用例】

```
MELSEC_ERR    err;

// CPUリセット
err = MELSEC_RemoteCpu(MELSEC_inst, &com_parm, MELSEC_REMOTE_RESET, TRUE);
```

MELSEC_GetCpuType

CPU形名の読み込み。

【関数仕様】

```
MELSEC_ERR      MELSEC_GetCpuType(MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm,
                                     MELSEC_CPU_TYPE* cpu_type, BOOL log_enable);
```

inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
cpu_type	CPU 形名格納ポインタ
log_enable	通信ログ書き込み有無（コールバック） =TRUE: 行う =FALSE: 行わない

戻り値	エラー番号（=0：正常完了、≠0：異常完了）
-----	------------------------

【CPU形名構造体】

```
typedef struct {
    WORD    Code;           // 形名コード
    TCHAR   Name[32];       // 形名
} MELSEC_CPU_TYPE;
```

【関数使用例】

```
MELSEC_CPU_TYPE  cpu_type;
MELSEC_ERR       err;

// CPU形名読み込み
err = MELSEC_GetCpuType(MELSEC_inst, &com_parm, &cpu_type, TRUE);

// Q02HCPU の場合、下記値がセットされる。
cpu_type.Code    0x0041
cpu_type.Name    "Q02HCU"
```

MELSEC_LoopbackTest

ループバック通信テストを行う。

【関数仕様】

```
MELSEC_ERR      MELSEC_LoopbackTest(MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm,  
                                       WORD cnt, LPCTSTR data, BOOL log_enable);
```

inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
cnt	ループバックデータ数 (1~254)
data	ループバックデータポインタ データは、ASCII コード文字のみとする。
log_enable	通信ログ書込み有無 (コールバック) =TRUE: 行う =FALSE: 行わない
戻り値	エラー番号 (=0 : 正常完了、≠0 : 異常完了)

【関数使用例】

```
MELSEC_ERR      err;  
  
// ループバックテスト  
err = MELSEC_LoopbackTest(MELSEC_inst, &com_parm, 5, "ABCDE", TRUE);
```

MELSEC_ReadArrayLabel

ラベル名によるワードデバイスまたはビットデバイス状態を読み込む。

【関数仕様】

MELSEC_ERR	MELSEC_ReadArrayLabel (MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm, LPCTSTR label, WORD label_type, WORD cnt, WORD* data, BOOL log_enable);
inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
label	ラベル名が格納されている文字列の先頭ポインタ (ラベル名は、最大 50 文字で、ラベル名の最後は、NULL 文字であること)
label_type	ラベルデバイスタイプ = MELSEC_LABEL_TYPE_BITS : ビット定義ラベル = MELSEC_LABEL_TYPE_WORDS : ワード定義ラベル
cnt	ラベル読み込み配列数
data	読み込みデータを格納する先頭ポインタ デバイス番号がビットデバイスの場合、 ON/OFF状態は、下位ビットからセットされる。
log_enable	通信ログ書込み有無 (コールバック) =TRUE: 行う =FALSE: 行わない
戻り値	エラー番号 (=0 : 正常完了、≠0 : 異常完了)

【関数使用例】

```
WORD          dev_data[256];
MELSEC_ERR    err;

// 指定したラベル配列 5 (sample_label[5]) から 10 点の読み込み (ラベルがワード定義の場合)
err = MELSEC_ReadArrayLabel (MELSEC_inst, &com_parm, "sample_label[5]", MELSEC_LABEL_WORDS,
                             10, dev_data, TRUE);
```

MELSEC_WriteArrayLabel

ラベル名によるワードデバイスまたはビットデバイスへ書き込む。

【関数仕様】

MELSEC_ERR	MELSEC_WriteArrayLabel (MELSEC_INSTANCE inst_no, MELSEC_COM_PARM* parm, LPCTSTR label, WORD label_type, WORD cnt, WORD* data, BOOL log_enable);
inst_no	初期化したインスタンス番号
parm	通信パラメータポインタ NULL ポインタの場合は、デフォルトパラメータ値が使用される。
label	ラベル名が格納されている文字列の先頭ポインタ (ラベル名は、最大 50 文字で、ラベル名の最後は、NULL 文字であること)
label_type	ラベルデバイスタイプ = MELSEC_LABEL_TYPE_BITS : ビット定義ラベル = MELSEC_LABEL_TYPE_WORDS : ワード定義ラベル
cnt	ラベル書き込み配列数
data	書き込みデータが格納されてる先頭ポインタ デバイス番号がビットデバイスの場合、 ON/OFF状態は、下位ビットからセットされていること。
log_enable	通信ログ書き込み有無 (コールバック) =TRUE: 行う =FALSE: 行わない
戻り値	エラー番号 (=0 : 正常完了、≠0 : 異常完了)

【関数使用例】

```
WORD          dev_data[256];
MELSEC_ERR    err;
LONG          i;

// ワードデバイスの書き込み (D123 から 10 ワード書き込み)
for (i = 0; i < 10; i++) {
    dev_data[i] = (WORD)i;
}

// 指定したラベル配列 5 (sample_label[5]) から 10 点の書き込み (ラベルがワード定義の場合)
err = MELSEC_WriteArrayLabel(MELSEC_inst, &com_parm, "sample_label[5]", MELSEC_LABEL_WORDS,
                              10, dev_data, TRUE);
```